C – The Pointer Book by Klaus Schröder

Supervised by: Feim Ridvan, RASIM

M.Sc Alperen BÜYÜKATAK in Information and Communication Technologies

1. Data Types

1.1 Conceptual Foundations of Data Types in the C Programming Language

Digital computer systems operate on binary information. At the lowest level, all data is represented as sequences of bits, while memory is typically accessed in units of bytes consisting of eight bits. This byte-based organization forms the fundamental structure for data storage and processing in modern computer architectures.

From the hardware perspective, memory itself has no inherent meaning. A sequence of bits stored at a given location does not automatically represent an integer, a character, or any other value. Its interpretation depends on how the programming language and compiler define and access the stored data.

High-level programming languages introduce data types to provide this interpretation. In the C programming language, data types specify both the amount of memory required for an object and the way the stored bits are interpreted during program execution. Because C is closely connected to the underlying hardware, certain properties—such as the size of basic types—may depend on the compiler and target platform.

Understanding the role of data types is essential when working with low-level software, embedded systems, or performance-critical applications, where precise control over memory representation and interpretation is required.

1.2 The Integer Type int and Its Representation

Among the fundamental data types in C, int has a central role. It is intended for representing integer values efficiently according to the target machine and is commonly used for general-purpose arithmetic. The language standard guarantees only a minimum range of values and leaves the exact size of int to the compiler implementation.

In most systems, int is a signed type, meaning that it can represent both positive and negative values. At the binary level, signed integers are commonly represented using two's complement. In this representation, the most significant bit acts as the sign bit: if it is zero, the value is non-negative; if it is one, the value is negative.

Unsigned

8	4	2	1
1	1	1	1

8+4+2+1 = 15

Signed

sign	4	2	1
0	1	1	1

4+2+1 = 7

Figure 1.2.1. *Unsigned vs. signed integer representation.*

For an n-bit signed integer, the representable range is from $-2^{n-1}$to $2^{n-1}-1$. This range is asymmetric, since the most negative value has no positive counterpart of equal magnitude. The all-zero bit pattern represents zero.

An important consequence of this representation is integer overflow. When the result of an operation exceeds the representable range, the excess bits are discarded at the hardware level. In C, signed

overflow is not detected by default and may lead to undefined behavior. For this reason, programmers must avoid relying on overflow behavior in portable code.

Type	Size(bytes)
char	1
short	2
int	2/4
long	4
float	4
double	8

Figure 1.2.2. *Typical sizes of fundamental C data types expressed in bytes.*

Note. The exact size of these data types is platform and compiler dependent.

Since the exact size of int depends on the compiler and platform, programs should not assume a fixed bit width. Instead, portability requires careful use of language constructs such as sizeof and, where necessary, the introduction of application-specific type definitions.

1.3 Floating-Point Data Types and Numerical Representation

While integer types represent discrete values, many scientific and engineering applications require the representation of real numbers. In C, such values are handled through the floating-point types float, double, and long double. Unlike integers, floating-point numbers use a structured representation that separates sign, exponent, and mantissa, allowing a much wider numerical range.

Floating-point values are commonly implemented according to the IEEE 754 standard. This representation makes it possible to store very small and very large numbers, but it also introduces limitations in precision. Since many decimal values cannot be represented exactly in binary form, floating-point calculations are subject to rounding errors that may accumulate over time.

sign	exponent	mantissa
0	10000110	11010100000000000000000
1bit	8bit	23bit

32bit

Figure 1.3. *IEEE 754 single-precision floating-point bit layout (sign, exponent, mantissa).*

The mantissa represents the significant digits of the number, while the exponent determines the scale by specifying a power of two. This structure allows floating-point types to represent numbers across a very wide range of magnitudes, far exceeding what is possible with fixed-width integer types. However, this flexibility comes at the cost of complexity, both in representation and computation.

The three floating-point types mainly differ in precision and storage size. In general, float provides lower precision, double offers greater accuracy, and long double extends precision further depending on the compiler and platform. As a result, the choice of type affects both numerical accuracy and memory usage.

Floating-point operations are usually more expensive than integer operations. On systems with hardware support, they are executed efficiently, whereas on systems without such support they may be performed in software, which reduces performance. For this reason, the use of floating-point types always involves a balance between numerical range, precision, and computational cost.

Understanding these characteristics is essential when developing reliable and efficient C programs, particularly in embedded or performance-sensitive applications.

1.4 Type Definition and Portability Using typedef

One of the defining characteristics of C is the flexibility of its type system. Although this flexibility allows efficient use of hardware resources, it can also create portability problems when software is moved across different compilers or platforms. Variations in data type sizes and memory organization may lead to subtle errors if they are not carefully managed.

To reduce such problems, C provides the typedef mechanism, which allows programmers to define alternative names for existing data types. Rather than using built-in types such as int or long directly throughout a program, application-specific names can be introduced to reflect the intended use of the data.

This approach is particularly useful when software interacts with hardware registers, communication formats, or other data objects whose size must remain conceptually stable across systems. If the underlying platform changes, only the type definition needs to be updated, while the rest of the program can remain unchanged.

In addition to portability, typedef improves readability by making the purpose of a variable clearer than primitive types alone. Although it does not create a new type in the strict sense, it provides a practical abstraction that supports more maintainable and portable C programs.

2. Access to Data

2.1 Declaration

Before any data object can be accessed in a C program, it must first be made known to the compiler. This is achieved through a declaration, which associates a name with a data type and thereby defines how the object is to be interpreted during translation.

A declaration introduces the existence and type of a variable or function without necessarily allocating storage for it. Its purpose is to provide the compiler with sufficient information for type checking, expression evaluation, and function call verification. In this sense, a declaration defines how an identifier is to be treated even if its actual memory location is determined later.

Function prototypes represent an important form of declaration. By specifying the return type and parameter types of a function, a prototype enables the compiler to verify function calls before the full function definition is encountered. This reduces the risk of type-related errors and improves consistency across separately compiled program units.

2.2 Definition

While a declaration introduces a name and its type, a definition additionally allocates storage for the corresponding object. A definition therefore establishes the concrete existence of a variable or function within the program.

For variables, a definition reserves the required memory and determines the size of the object according to its data type. In relocatable programs, the final absolute address is not fixed during source translation but is resolved later by the linker and the loader. Even so, the definition is the point at which the object becomes a real storage entity within the program.

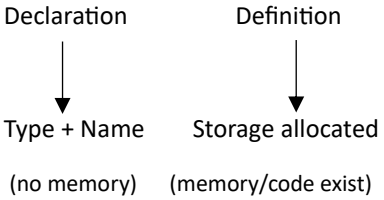

Figure 2.2. *Distinction between declaration and definition in C.*

The distinction between declaration and definition is especially important in multi-file programs. A variable may be declared in several places but must be defined only once. This separation enables modular programming and allows multiple program units to share common objects without duplicating storage.

2.3 Direct Addressing of Data

Once a variable has been declared and defined, it can be accessed directly by its name. In this form of access, the identifier represents the value stored at the associated memory location.

Direct addressing is the standard mode of variable access in C. It is used both when assigning values and when evaluating expressions. Because the compiler already knows the type, size, and storage properties of the variable, it can generate the required instructions without additional information from the programmer.

The advantage of direct addressing lies primarily in clarity. It produces readable source code and allows ordinary program logic to be expressed without exposing the details of memory access. For this reason, direct addressing remains the dominant form of access in most C programs.

2.4 Implicit Type Conversion

One of the distinguishing characteristics of C is that expressions may contain operands of different data types. In such cases, the language performs implicit type conversion in order to make the operands compatible before the operation is executed.

Lower Rank	Promoted To
char	int
short	int
int	float
int	double

Figure 2.4. *Typical implicit type promotions performed during arithmetic expressions in C.*

This conversion follows a hierarchy in which lower-ranked types are promoted to higher-ranked types. For example, smaller integer types such as char and short are typically promoted to int, while integer values are converted to floating-point representation when they appear together with floating-point operands. The purpose of this mechanism is to ensure consistent evaluation of expressions.

Although implicit conversion provides convenience, it may also introduce problems. Precision may be lost, signs may change unexpectedly, and runtime overhead may arise, especially in floating-point expressions. As a result, programmers must understand the conversion rules in order to avoid subtle errors and unintended behavior.

2.5 Access to Return Values of Functions

Functions in C are not only used to structure program logic but also to produce values. The return value of a function behaves like a temporary unnamed object whose type is defined by the function's return type.

A function call may therefore be used in the same way as a variable or constant. Its return value can be assigned to another variable, included in an arithmetic expression, or passed directly as an argument to another function. In each case, the returned value is handled according to the ordinary type rules of the language.

This behavior makes function return values a natural part of expression evaluation. At the same time, it emphasizes the importance of specifying function return types correctly, since truncation, rounding, or loss of precision may otherwise occur when the return value is used in a context expecting another type.

2.6 Explicit Type Conversion

When implicit conversion does not produce the intended result, C allows the programmer to force a specific conversion through the cast operator. Explicit type conversion is written by placing the target type in parentheses before the value or expression to be converted.

Aspect	Implicit Conversion	Explicit Conversion
Who performs the conversion	Compiler	Programmer
Syntax	Automatic	(type) cast
Programmer control	Limited	Full
Risk of unintended effects	Higher	Lower (intentional)
Typical usage	Expressions	Critical conversions

Figure 2.6. *Comparison between implicit and explicit type conversion in C.*

A cast overrides the default conversion rules and specifies how the value is to be interpreted in the surrounding expression. This is particularly useful when a specific precision level, signedness, or numeric format is required. It is also important when dealing with function return values, mixed-type expressions, or constants whose default type is not suitable for the intended use.

Explicit conversion improves control but does not automatically make a program safer. If a cast causes truncation, overflow, or loss of precision, these effects remain the programmer's responsibility. For this reason, explicit conversions should be used carefully and only when their consequences are well understood.

2.7 Indirect Addressing

In direct addressing, a variable name is used to access its value. In indirect addressing, the address of the variable is used instead, and the stored value is accessed through that address. This mechanism forms the basis of pointer-based programming in C.

Indirect addressing is realized through the address-of operator & and the dereference operator *. The first produces the memory address of an object, while the second accesses the value stored at a given address. Together, these operators establish a direct relationship between variables, addresses, and stored data.

Indirect addressing is essential whenever the identity of the accessed object is not known until runtime. It is therefore widely used in pointer manipulation, dynamic memory management, traversal of arrays and structures, and access across function boundaries. Although more complex than direct addressing, it provides a much greater degree of flexibility.

2.8 Absolute Addressing

In some programming situations, particularly in embedded systems and low-level hardware programming, data must be accessed not through ordinary variables but through fixed memory locations. This form of access is referred to as absolute addressing.

Absolute addressing uses a specific numerical address to reach a memory location or a hardware register. Unlike ordinary program variables, such addresses are not determined by the compiler as part of relocatable program storage, but correspond to predefined positions in the processor's address space.

Because C does not provide a special syntax for this purpose, absolute addressing is achieved by converting a constant address into a typed pointer and then dereferencing it. This allows the programmer to impose a specific data interpretation on a raw address and access the referenced object accordingly. Absolute addressing is therefore an essential technique for memory-mapped I/O and other hardware-oriented applications.

2.9 Explicit Type Conversion of Addresses

When addresses are used directly in C, they must be associated with a data type before meaningful access is possible. A numerical address alone does not specify whether the referenced object should be interpreted as a byte, a word, or a larger structure.

For this reason, explicit type conversion of addresses is required. By casting an address to a pointer of a particular type, the programmer defines both the size of the access and the interpretation of the retrieved bit pattern. Without such a conversion, the compiler cannot generate correct memory access instructions.

This mechanism is particularly important in absolute addressing. The cast does not change the address itself; it changes only how the compiler interprets and accesses the memory at that location. Correct type conversion therefore ensures that memory is accessed in a way that matches the intended data representation.

3. Pointer

In the previous chapters, data types, declarations, definitions, and addressing mechanisms were introduced. These concepts explain how variables are stored, interpreted, and accessed in memory. Building on this foundation, this chapter introduces pointers as variables that store memory addresses.

A pointer differs from an ordinary variable in that its value is not data itself, but the address of data. While a variable name provides direct access to its stored value, a pointer allows that address to be stored, transferred, and used to access the same memory location indirectly. In this way, addresses become manipulable program objects.

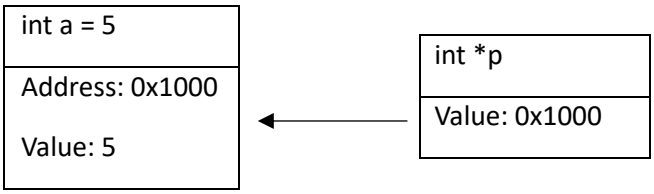

Figure 3.0. *Relationship between variable, address, and pointer*

The use of pointers greatly increases the expressive power of C. They enable dynamic memory management, parameter passing by reference, flexible data structures, and interaction with hardware. At the same time, pointer-based programming requires a precise understanding of memory layout, object lifetime, and type interpretation. For this reason, incorrect pointer usage can easily lead to undefined behavior.

3.1 Pointer to void

Among the different forms of pointers in C, the pointer to void has a special role. In this context, void denotes the absence of a specific data type, so a void* represents an untyped pointer that can hold the address of any data object.

A pointer to void can be assigned the address of a variable regardless of its type, without requiring explicit conversion. This makes void* a general-purpose carrier for addresses and allows functions to operate on memory when the concrete type is not yet known. Such flexibility is particularly useful in generic programming and in memory management routines.

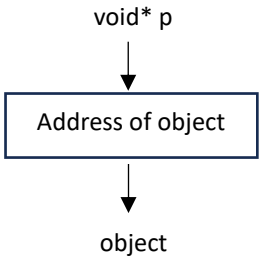

Figure 3.1. *A void pointer as a type-agnostic reference to memory.*

However, a void* cannot be used directly to access stored data. Since the compiler does not know the size or structure of the referenced object, dereferencing a void* is not meaningful until it has been converted to an appropriate typed pointer. In this sense, void* is mainly an intermediate form for transporting addresses rather than a complete access mechanism.

3.2 Null Pointer

A pointer in C may, in principle, contain any address value. Whether that value refers to a valid memory location depends on the memory layout of the system and the execution context of the program. To distinguish valid and invalid pointer states, C defines the concept of the null pointer.

A null pointer is a pointer that does not refer to any valid object or memory location. It is typically represented by the constant zero, although its exact internal representation is implementation-defined. Conceptually, it serves as a sentinel value indicating the absence of a usable address.

This makes the null pointer important for initialization and error handling. Pointers are often initialized to the null value before receiving a meaningful address, and functions that return pointers commonly use it to signal failure. For objects with static or external storage duration, the absence of an explicit initializer results in automatic initialization to the null pointer.

The null pointer therefore provides a well-defined and portable way to represent “no address,” making pointer-related program states easier to test and control.

3.3 Pointer as Return Value of a Function

Functions in C may return a single value, and this value may also be a pointer. In that case, the function returns an address that refers to a memory location accessible to the caller.

The return type of the function specifies the type of object to which the returned pointer refers. From the caller’s point of view, the result of the function can be assigned to a pointer variable, dereferenced to access data, or passed to another function. It is therefore handled like any other pointer value.

This mechanism is particularly important in dynamic memory management. A function may request memory from the system and return the starting address of the allocated block. The caller does not need to know where the memory is located physically; it only needs a valid pointer through which the memory can be accessed.

However, the validity of the returned pointer depends on the lifetime of the referenced object. Returning the address of an automatic local object is unsafe, since that object ceases to exist when the function returns. For this reason, pointer-returning functions should return addresses of static objects, dynamically allocated memory, or objects whose lifetime extends beyond the function scope.

3.4 Access with Pointers

Accessing data through pointers is a natural extension of the addressing mechanisms introduced earlier. While direct addressing uses the name of a variable to obtain its value, pointer-based access uses the address stored in a pointer variable. This access is performed through the dereference operator *, which denotes the value located at the referenced address.

When accessing data through a pointer, it is important to distinguish between the pointer value itself and the value stored at the address to which it refers. In this sense, pointer access corresponds closely to absolute addressing, provided that the address has been correctly typed. Once a raw address has been converted into a suitable pointer type, access through the pointer and access through the typed address are equivalent in effect.

The meaning of the * operator depends on its position in the source code. In a declaration, it means “pointer to”; in an expression, it refers to the value addressed by the pointer. Although this dual role can initially seem confusing, it reflects the close relationship between addresses and pointers in C.

As pointer expressions become more complex, especially when several levels of indirection are involved, interpretation becomes more difficult. A useful principle is that if the part corresponding to the declaration is removed, the remaining form indicates the type of the accessed value. This helps reduce complex pointer expressions to their essential meaning.

3.5 Pointer Arithmetic

Pointer arithmetic is defined with respect to the data type to which the pointer refers. Although pointer values are numerically represented as addresses, arithmetic operations on pointers are not interpreted as simple byte-wise calculations. Instead, they are scaled according to the size of the referenced type.

The addition or subtraction of an integer value to or from a pointer produces a new pointer that refers to a memory location displaced by a corresponding number of elements of the referenced data type. The displacement in bytes is computed as the integer value multiplied by sizeof() of the referenced type. Consequently, incrementing a pointer advances it by one element of that type rather than by one byte. This definition ensures consistency with the layout of arrays and contiguous memory regions.

The increment and decrement operators are defined analogously. However, due to operator precedence and potential side effects, expressions that combine dereferencing and modification operations must be interpreted strictly according to the language specification.

Multiplication and division of pointer values are not defined. Likewise, addition of two pointers is undefined because it does not correspond to a meaningful memory access operation. Subtraction of two pointers is permitted only when both refer to elements within the same array object and have compatible types. In such cases, the result represents the number of elements separating the two positions.

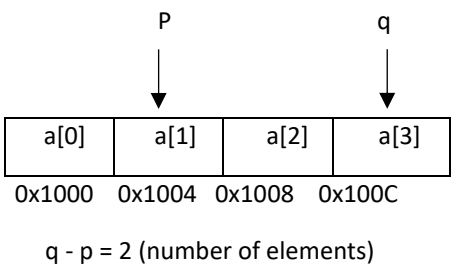

Figure 3.5. *Valid pointer subtraction within the same array*

C does not perform runtime validation of pointer arithmetic. The language does not verify whether the resulting pointer refers to a valid object or allocated memory region. Incorrect pointer calculations may therefore result in undefined behavior. For this reason, pointer arithmetic is powerful but must be used with particular care.

4. One-Dimensional Data Fields

A one-dimensional data field in C, commonly called an array, is a finite sequence of elements of the same type stored under one identifier. Unlike scalar variables, arrays represent ordered collections of values. Their elements are stored in contiguous memory locations, which makes address calculation predictable and efficient.

The declaration of an array specifies its element type and size. In static declarations, this size must be known at compile time. Since all elements share the same type and are placed consecutively in memory, the address of each element is derived from the base address of the array and the corresponding index. Array indices begin at zero, and C does not perform automatic bounds checking. Therefore, invalid index use may cause undefined behavior.

Index (n)	Address (int=4B)
0	0x1000
1	0x1004
2	0x1008
3	0x100C

Table 4.1. *Example of index-based address computation.*

The name of an array represents the address of its first element. However, unlike a pointer variable, this address is fixed and cannot be reassigned. For this reason, arrays are closely related to pointers in use, but they are not identical in meaning.

4.1 Pointer to an Array

Array indexing and pointer arithmetic are based on the same address calculation. A pointer initialized with the address of the first array element refers to the same memory region as the array name. The important difference is that the pointer may later be changed, while the array address remains fixed.

For correct access, the pointer type must match the array element type. Pointer-based access therefore does not introduce a different memory model, but an alternative way of referring to the same array structure.

4.2 Array Address as a Function Argument

When an array is passed to a function in C, the entire array is not copied. Instead, only the address of its first element is transferred. The function therefore accesses the original memory region directly.

Because only the base address is passed, the function cannot determine the number of elements automatically. If size information is needed, it must be supplied separately. Inside the function, element access may be performed either by indexing or by pointer arithmetic.

4.3 Array Address as Return Value of a Function

Just as an array address can be passed to a function, a function may also return the address of an array element. In practice, however, a function does not return an entire array as a value; it returns a pointer to the element type. This pointer represents the base address of a memory region containing array data.

When returning an address, particular attention must be paid to object lifetime. If the referenced array was defined within the function using automatic storage duration, its memory is released when the function terminates. Returning the address of such a local array results in undefined behavior, since the pointer refers to memory that is no longer valid.

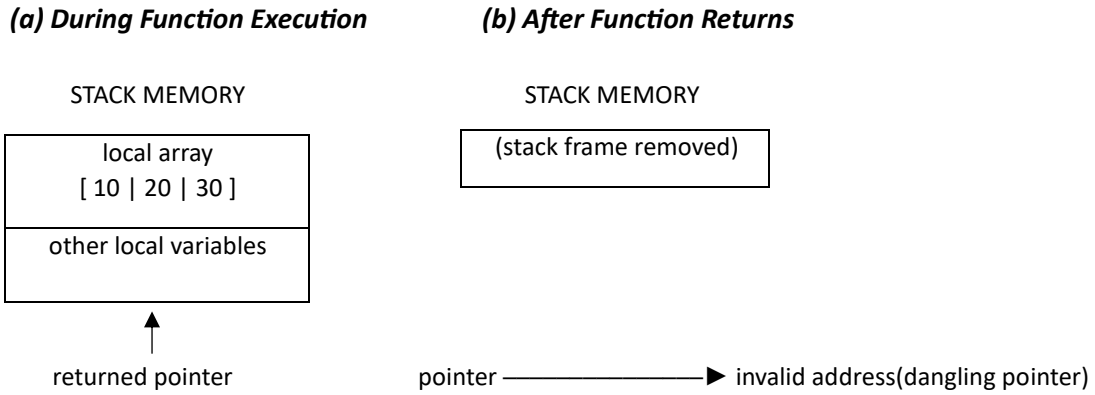

Figure 4.3. *Returning a local array address creates a dangling pointer.*

Two critical considerations therefore arise when a function returns a pointer to array data. First, the pointer itself does not carry information about the size of the array. Any required size information must be managed separately. Second, the validity of the returned address depends entirely on the storage duration of the referenced object. Only memory regions that remain valid after function termination, such as dynamically allocated memory or statically defined objects, may be safely returned.

If these constraints are not respected, the returned address may temporarily appear valid but can later lead to unpredictable behavior once the stack memory is reused. Such errors are often difficult to detect, as they depend on runtime conditions rather than deterministic compile-time checks.

4.4 Initialization of Data Arrays

Arrays may be initialized at the time of declaration by listing their initial values. If fewer values are specified than the declared size, the remaining elements are automatically initialized to zero. This ensures a well-defined initial state for the whole array. In many cases, the array size may be omitted when it can be derived from the number of initialization values. This reduces redundancy and allows the compiler to determine the required size automatically.

4.5 Initialization of Strings

A character array may represent a string. In C, however, a string is not a distinct built-in type, but a sequence of characters terminated by a null byte. This terminating character is essential because it marks the end of the string and allows functions to determine where the character sequence stops.

In memory, a string is stored like any other array: as a contiguous sequence of elements. The only conceptual distinction lies in the presence of the terminating null byte, which enables functions to determine where the string ends. Without this terminator, the character sequence cannot be safely interpreted as a string.

A string may consist solely of the null character, representing an empty string. Importantly, the null byte cannot appear as an internal character of a valid string, since it marks termination.

String: "abc"

'a'	'b'	'c'	'\0'
0x2000	0x2001	0x2002	0x2003

'\0' = null terminator (end of string)

Figure 4.5. *Memory representation of a null-terminated string.*

Apart from their termination rule, strings behave like ordinary character arrays. They can be accessed element by element and manipulated through indexing or pointer-based operations. Their special status arises not from their memory structure, but from the semantic interpretation imposed by string-handling conventions.

4.6 Initialization of Pointers to a Data Array

A pointer may be initialized with the address of an array, so that both the pointer and the array name initially refer to the same memory region. This is a common and useful technique, especially when arrays are to be processed through pointer-based operations rather than explicit indexing.

Even in this case, the difference between arrays and pointers remains fundamental. The array identifier is tied permanently to its storage region, whereas the pointer may later be redirected to another address. Reassigning a pointer therefore changes only the reference, not the array itself. This distinction becomes especially important in string handling and general memory management, where flexible references are often useful but fixed storage objects must still be clearly distinguished from them. Understanding this difference is essential for correct reasoning about addresses, storage, and data access in C.

5. Multidimensional Data Arrays

In many applications, one dimensional arrays are not sufficient to represent structured data. C therefore provides multidimensional arrays, which extend the array concept to multiple indices while preserving the same linear memory organization. A multidimensional array can be understood as an array whose elements are themselves arrays. In the two dimensional case, this structure is typically interpreted as rows and columns. Although this representation appears rectangular, the physical memory layout remains linear, since all elements are stored consecutively in a single contiguous block. In such arrays, the first index selects the row and the second index selects the element within that row. Because elements are stored row by row in memory, this ordering determines the calculation of element addresses.

Row 0 →	a[0][0]	a[0][1]	a[0][2]
Row 1 →	a[1][0]	a[1][1]	a[1][2]

Memory order:
a[0][0]
a[0][1]
a[0][2]
a[1][0]
a[1][1]
a[1][2]

Figure 5.0. *Row-major memory ordering in C.*

The general principles known from one dimensional arrays still apply, including contiguous storage, zero based indexing, and the absence of automatic bounds checking. Additional dimensions therefore influence only the interpretation of indices and the resulting address calculation.

5.1 Two-Dimensional Data Arrays

A two dimensional array represents a collection of rows, where each row itself is a one dimensional array. The entire structure therefore consists of several arrays stored consecutively in memory.

Access to an element requires two indices. The first index selects the row, and the second index selects the column within that row. Both indices begin at zero.

Initialization may be performed row by row during declaration. If fewer values are specified than required, the remaining elements are initialized to zero. For correct memory layout, the number of columns must always be specified because it defines the size of each row.

The address of an element is derived from the base address of the array and an offset determined by the row and column indices. Although the structure appears two dimensional, the actual storage remains linear in memory.

Two dimensional arrays are commonly used for matrices, tables, and other structured datasets.

a[0][0]	a[0][1]	a[0][2]
1000	1004	1008
a[1][0]	a[1][1]	a[1][2]
1012	1016	1020
a[2][0]	a[2][1]	a[2][2]
1024	1028	1032

Figure 5.1. *Contiguous row-major memory layout of a two-dimensional data array*

Two dimensional arrays are particularly suited for matrix representations, structured text layouts, and similar rectangular data models. Their conceptual form is multidimensional, yet their memory organization remains contiguous and linear.

5.2 String Arrays

In C, a string is represented as a character array terminated by a null byte. When multiple strings are stored under a single identifier, they are often organized as a two dimensional character array. In this arrangement each row represents one string, while the columns represent the character positions within that string.

The column dimension defines the maximum string length including the terminating null character. Each row therefore reserves the same amount of memory even if the actual string lengths differ. Rows may be initialized using string literals, and if fewer rows are specified than declared, the remaining rows are filled with null characters.

Row 0 →	J	o	h	n	\0	?

(? = unused memory (padding inside row))

Row 1 →	E	m	m	a	\0	?

Linear Memory (Actual Layout):

J o h n \0 ? E m m a \0 ?

Figure 5.2. *Two-dimensional character array storing null-terminated strings in fixed-width rows.*

Accessing a row yields the starting address of the corresponding character sequence, while individual characters are accessed using the column index. This organization allows multiple strings to be stored within a predictable memory layout while preserving the null terminated string convention used in C.

5.3 n-Dimensional Data Fields

The principles established for two dimensional arrays extend naturally to arrays of arbitrary dimension. An n dimensional array is defined as an array whose elements are themselves arrays of dimension n minus one. Despite the increasing conceptual complexity, the storage model remains unchanged.

Multidimensional arrays may be manipulated in two principal ways. The first method relies on explicit indexing, where each dimension is addressed through its corresponding index. When indices are applied consistently, this approach provides a clear and structured representation of the data field.

The second method uses pointer based addressing. In this case, a pointer to the base type is used without reference to the declared dimensions. Pointer arithmetic interprets the structure as a linear sequence of elements and advances according to the size of the base type rather than the conceptual dimensions.

If fewer indices are specified than the array possesses, the expression yields the address of a subordinate data field rather than a complete element. The omitted dimensions correspond to a remaining lower dimensional array whose base address is produced by the expression. This behavior reflects the hierarchical definition of multidimensional arrays as nested array structures.

5.4 Determining the Size of Multidimensional Data Fields

The total memory occupied by a data field is determined by the number of elements multiplied by
The total memory occupied by a data field depends on the number of elements multiplied by the size

of the element type. For multidimensional arrays this corresponds to the product of all dimension sizes and the size of a single element. Memory layout may also be influenced by alignment requirements imposed by the compiler, since certain data types must begin at addresses divisible by specific byte boundaries.

The operator sizeof provides a reliable mechanism for determining object size in bytes. When applied to an array it returns the total memory size of the entire structure, while applying it to a row or element yields the size of the corresponding substructure. Dividing the total array size by the size of one element allows the number of stored elements to be determined. Using sizeof directly on program objects ensures correct results even if data types or dimension declarations change.

5.5 Pointer Addressing of Multidimensional Data Fields

Pointer addressing allows multidimensional arrays to be processed according to their underlying linear memory layout. Although these arrays are conceptually organized into rows and columns, pointer arithmetic advances through contiguous memory according to the size of the referenced type.

Correct pointer declarations must reflect the structure of the referenced subfield. For example, a pointer to a row of a two dimensional array advances by the size of an entire row rather than by the size of a single element. If fewer indices are used than declared dimensions, the resulting expression refers to a subordinate array whose address may then be dereferenced.

In practice, explicit index based addressing is often clearer and less error prone, while pointer based access provides flexibility for navigating structured memory. Correct handling of multidimensional arrays therefore requires careful matching between pointer types and array structure to avoid incorrect address calculations.

6. Pointer Fields

In many applications, the elements of data fields represent actual data values of a specific type. However, it is also possible to construct fields whose elements are pointers. When multiple pointers are grouped under a single identifier, the resulting structure is called a pointer field. In such a structure, the elements do not contain data values themselves but store addresses that refer to data located elsewhere in memory.

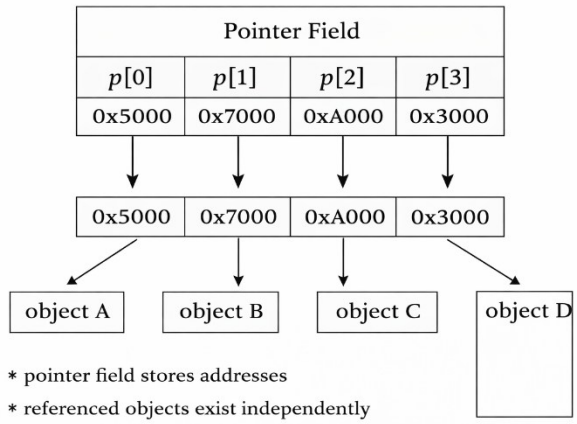

Figure 6.0. *Pointer field stores addresses;referenced objects are stored independently*

A pointer field may be one dimensional or multidimensional. Each element stores a memory address, while the referenced objects may differ in structure, size, or location. An important distinction is that memory is allocated only for the pointer elements themselves. The data objects referenced by these pointers must exist independently. Consequently, the size of a pointer field depends only on the number of pointer elements and the size of a pointer in the given system environment.

6.1 One Dimensional Pointer Fields

A one dimensional pointer field represents a sequence of pointers defined using the pointer operator together with the index operator. Conceptually, the structure is similar to an ordinary array, but its elements contain addresses rather than direct data values.

Each pointer element may refer to a variable, an array element, or the starting address of a larger data structure. Access to the referenced data may occur through pointer dereferencing or through index notation when the pointer refers to an array. Since the elements store only addresses, initialization assigns valid addresses of compatible objects rather than the actual data values.

Pointer arithmetic operates relative to the pointer's base type. Advancing a pointer therefore increases the address according to the size of the referenced type rather than the size of the referenced object. As a result, a one dimensional pointer field represents a collection of references that connect the array structure with data objects stored elsewhere in memory.

6.2 Pointer Arrays for String Addressing

When working with collections of text strings, two different memory organizations are common. One approach uses a two dimensional character array in which each row stores a string and each column represents a character position. In this representation, all strings occupy a single contiguous memory region with a fixed maximum length.

An alternative approach uses a pointer array in which each element stores the address of an independent character array. Each string is therefore stored separately, and the pointer field provides access to their starting addresses. Unlike the two dimensional character array, this structure does not require all strings to have the same length.

Because the strings are stored independently, pointer arrays allow flexible memory organization. Individual strings may be replaced, reassigned, or allocated dynamically without affecting the surrounding elements. However, pointer arithmetic must remain within the bounds of the referenced object. Crossing structural boundaries may lead to undefined behavior if the referenced memory does not belong to the same object.

The choice between a two dimensional character array and a pointer array therefore reflects a trade off between structural regularity and referential flexibility. While the first approach guarantees a fixed memory layout, the second allows independent allocation and modification of individual strings.

7. Structures

In C programming, arrays require that all elements belong to the same data type. In many practical situations, however, a single object must contain variables of different types. A structure provides this

capability. It is a user defined data type that groups related variables under a single name, even if they have different base types.

Each component of a structure, called a member, may have its own data type. All members must ultimately be based on fundamental types or previously defined structured types. Structures therefore do not introduce new primitive types; rather, they allow composite data objects to be constructed from existing ones.

Conceptually, a structure defines a fixed memory layout. It specifies which members exist and in which order they are arranged.

```
struct {   WORD day;

           BYTE month[10];

           WORD year;

        }  born;
```

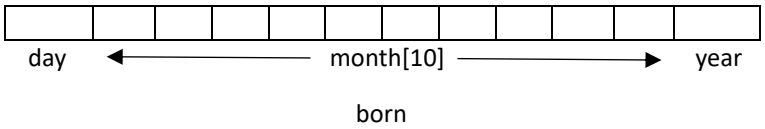

Figure 7.0. *Internal representation of a structure*

This layout determines how memory is allocated and how the individual components are accessed during program execution. Unlike arrays, which rely on indexed access and uniform element size, structures use named members whose sizes may differ. The memory representation follows the declared order of members. Structures thus provide a mechanism for modeling complex entities while preserving a precise and predictable relationship between logical data organization and physical memory layout.

7.1 Structure Declarations

A structure declaration begins with the appropriate keyword and defines the members that belong to the structure. The declaration specifies the components and the order in which they appear. At this stage, only the layout is described. No memory is allocated until a variable of that structure type is defined. From the compiler's perspective, a structure declaration functions similarly to the specification of a standard data type. It defines how memory will be organized when a variable of that type is created. Once such a variable is declared, storage is reserved according to the structure layout.

Each member of a structure is accessed by referencing the structure variable and selecting the desired component. The structure variable represents the complete object, while its members represent distinct storage regions within that object.

A structure definition may also include a symbolic name known as a tag. The tag allows the defined layout to be referenced elsewhere in the program without repeating its full description. This improves clarity and consistency, especially in larger programs.

For better readability and portability, structure definitions are often combined with type definitions. This approach separates the definition of a data layout from the declaration of objects and simplifies reuse across multiple source files.

A structure may contain other structures as members, which allows hierarchical data organization. However, a structure cannot directly contain an instance of itself, because that would create an incomplete and infinitely expanding type. In such cases, references such as pointers are used to represent recursive relationships.

Structure declarations therefore provide a systematic method for defining composite data types and organizing related information within a program.

7.2 Arrays of Structures

Structures can also be used as array elements. In this case, each array element is a complete structure object of the same type. Access is performed by selecting an array element first and then selecting the desired member within that element. The assignment of values therefore takes place on the individual members of each structure element. Arrays of structures may also be multidimensional.

7.3 Complete Assignment

A structure is treated as a single variable representing the combined value of all its members. Consequently, one structure can be assigned to another structure of the same type in a single statement, which copies all components of the object.

This behavior distinguishes structures from arrays, which cannot be assigned as complete objects. The same principle applies in function calls: a structure may be passed as an argument, and a function may return a structure as its result.

7.4 Initialization of Structures

The initialization of a structure follows the same principle as array initialization. All members are listed in the order in which they are defined in the structure description. Each value must correspond to the declared type of the respective member.

If a structure contains another structure as a member, its components are initialized as a nested block. If array members are not completely specified during initialization, the remaining elements are automatically set to zero. The same rule applies to the structure as a whole, ensuring that unspecified members are initialized with zero values.

These rules apply not only to single structure variables but also to one dimensional and multidimensional arrays of structures.

7.5 Size of Structures

The memory space occupied by a structure is determined by the sizes of its individual members and by possible padding inserted for alignment purposes. For correct addressing, certain data types require starting addresses that satisfy specific alignment constraints. To ensure this, the compiler may insert unused bytes between members.

As a result, the total size of a structure is not always equal to the simple sum of the sizes of its components. Instead, it depends on the alignment rules of the target platform and the order in which the members are declared.

The size of a structure can be determined using the appropriate operator, which returns the number of bytes required to store the complete object. This operator may also be applied to individual components.

The ordering of members can influence the total memory consumption. By arranging members carefully, unnecessary padding can sometimes be reduced, leading to more compact memory usage.

7.6 Addresses of Structures

Since a structure variable is treated like any other variable, its address can be determined using the address operator. In this sense, the name of a structure represents the starting address of the memory region allocated for that structure. The address of a complete structure may be used in assignments or passed to functions. However, directly assigning an address to a structure variable itself is generally not meaningful, because a structure represents a value object rather than a reference.

It is important to distinguish between the address of the entire structure and the address of one of its members. Each component of a structure occupies a specific position within the overall memory layout. The address of a member is therefore derived from the base address of the structure plus an offset corresponding to the position of that member. When computing such addresses, operator precedence must be considered. The member access operator has higher priority than the address operator. For this reason, parentheses may be required to ensure correct interpretation of expressions involving structure addresses.

If a member of a structure is itself an array, referencing that member may already yield an address representation. In such cases, the explicit address operator is not required, since arrays are represented by the address of their first element.

Overall, addressing structures reinforces the idea that a structure is a contiguous memory block whose internal organization is defined by its member sequence.

7.7 Pointers to Structures

Pointers may also be defined to reference structure variables. A pointer to a structure stores the address of a structure object and allows indirect access to its members.

Access through a structure pointer follows the same principle as access to ordinary pointers. The pointer must first refer to a valid structure object. Member access through the pointer is then performed using the appropriate operator designed for this purpose. This operator combines dereferencing and member selection into a single expression.

Pointer arithmetic may be applied when a pointer references elements of an array of structures. In such a case, incrementing the pointer advances it to the next structure element. The step size automatically corresponds to the size of the complete structure, not to the size of an individual member.

Functions may use both complete structures and pointers to structures. Passing a pointer avoids copying the entire structure and allows direct modification of the original object. Returning a pointer to a structure is also possible, provided that the referenced object remains valid after function termination.

Structures cannot directly contain an instance of their own type, as this would create an infinite definition. However, they may contain pointers to their own type. This mechanism enables the construction of linked data structures such as lists and trees, where elements are connected through references rather than through direct embedding.

Pointers to structures therefore extend the flexibility of structured data by separating object storage from object reference. They allow efficient data manipulation while preserving the logical organization defined by the structure layout.

7.8 Structure Type FILE

For file processing such as reading, writing, or deleting, the C language provides a standardized interface through the library commonly known as the standard input output library. From the programmer's perspective, a file appears as a data object that functions as a buffer between external storage and the executing program.

Internally, this buffering mechanism is managed through a structure type referred to as FILE. The actual implementation of this structure is system dependent and therefore not portable at the source code level. Its internal layout is not intended to be accessed directly by the programmer. Instead, it is handled through library functions that operate on file objects.

When a file is opened, a corresponding structure object is created and managed by the runtime system. Access to the file occurs indirectly through a pointer that references this structure. This pointer represents the file handle and is used in subsequent input and output operations.

Standard streams such as input, output, and error are automatically initialized before program execution begins. These streams are associated with predefined file objects that are ready for immediate use. Whether a file is buffered, unbuffered, stored on disk, or connected to an input output device is transparent to the program logic. The abstraction provided by the FILE structure ensures that file handling remains independent of the underlying hardware.

In summary, the FILE structure encapsulates the internal state required for file management and provides a uniform interface for file operations within the C environment.

7.9 Variants and Unions

A union, also referred to as a variant type, is syntactically similar to a structure but differs fundamentally in memory organization. While a structure allocates separate memory regions for each member, a union reserves a single shared memory area that is used by all its members.

Only one member of a union can hold a meaningful value at a given time. All members occupy the same memory location, and the size of the union corresponds to the size of its largest member. This design allows a single memory area to be interpreted in different ways during program execution.

Unions are useful when a data object may represent different types of information at different times, but never simultaneously. By reusing the same memory region, they enable efficient storage when alternatives are mutually exclusive.

However, since all members share the same memory space, the programmer must carefully control which member is currently valid. The language does not automatically track the active alternative. Incorrect interpretation of the stored data may lead to undefined behavior.

Thus, unions provide flexibility in data representation by allowing multiple interpretations of a single memory region, while requiring disciplined usage to maintain correctness.

7.10 Bit Fields

Components of structures or unions may be declared as bit fields. In this case, a member does not occupy an entire storage unit such as a full integer. Instead, only a specified number of bits is reserved within an integer type, allowing multiple logical values to be stored compactly inside a single machine word.

A bit field behaves syntactically like a normal structure member but represents only a defined group of adjacent bits. Individual bits cannot be indexed or addressed directly, and a bit field does not have its own independent address.

Bit fields are particularly useful when modeling hardware registers, control flags, or status words, where specific bits represent configuration values. They allow symbolic access to these segments without requiring manual masking and shifting operations.

The underlying storage unit of a bit field is typically an integer type, and the number of allocated bits must not exceed the width of that type. Consecutive bit fields may share the same storage unit if sufficient space remains; otherwise, a new unit is allocated. Bit ordering, packing, and alignment are implementation dependent, which limits portability across platforms and compilers.

Although a bit field may occupy only a few bits, its value participates in expressions according to the rules of its declared base type. Bit fields therefore provide a compact mechanism for representing structured binary data, but their platform dependent layout requires careful use in portable programs.

8. Dynamic Memory Management

In many C programs, the required data structures are known at compile time. In such cases, memory can be reserved using static arrays whose size is fixed before the program begins execution.

However, in many practical applications the amount of data that must be processed becomes known only during program execution. For example, when processing measurement data or handling database records, the number of values may vary and cannot be determined in advance. Reserving the maximum possible memory beforehand would therefore waste memory and may be inefficient.

To handle such situations, C provides dynamic memory management. Memory is requested from the operating system only when it is needed during program execution. The allocated memory is taken from the heap, which grows as additional requests are made.

Dynamic memory allocation is performed through functions of the standard library, mainly malloc, calloc, and free.

8.1 Function malloc

The function malloc requests a block of memory from the heap during program execution. The size of the requested memory is specified in bytes and is typically calculated using the sizeof operator to ensure portability.

If the allocation is successful, malloc returns a pointer to the beginning of the allocated memory block. This pointer has no inherent type, so the programmer must interpret the memory according to the intended data type. The allocated memory can then be accessed similarly to an array using index notation.

If the memory request fails, the function returns a null pointer. Therefore, programs should always check whether the allocation was successful before using the memory. Additionally, the original pointer returned by malloc must be preserved, since it is required later to release the allocated memory.

8.2 Function calloc

The function calloc performs memory allocation similar to malloc, but with automatic initialization. The allocated memory region is set to zero before it is returned to the program.

Another difference lies in the parameters. Instead of specifying the total size directly, calloc receives two arguments: the number of elements and the size of each element. The total required memory is determined by multiplying these values. Because of this initialization behavior, calloc is often used when allocating arrays whose elements should initially contain zero values.

8.3 Function realloc

The function realloc is used to change the size of a memory block that was previously allocated with malloc or calloc. If the required memory size changes during program execution, realloc allows the allocated memory area to be enlarged or reduced.

The pointer returned by a previous allocation is passed as the first argument. The second argument specifies the new memory size in bytes. If the existing memory block cannot be extended at its current location, a new memory area is allocated and the existing data is copied to this new region. In such a case, the old memory block must be released afterwards. If the allocation fails, the function returns a null pointer.

8.4 Function free

Memory that has been allocated dynamically using malloc, calloc, or realloc must eventually be returned to the system. This is done using the function free.

The pointer passed to free must refer to a memory region that was previously allocated by one of these functions. Passing any other pointer results in undefined behavior. When memory is released, it becomes available again for future allocation requests. However, separately freed memory blocks are not automatically merged into one larger block.

9 Linked Lists

When managing larger data sets, representing data solely in the form of arrays can become limiting. Arrays require that the number of elements be known in advance and that all elements be stored in contiguous memory locations. In many applications, however, the amount of data changes dynamically during program execution. In such cases, array-based data management becomes inefficient or impractical.

Dynamic memory management provides the possibility of allocating memory during runtime. Nevertheless, arrays remain unsuitable for situations in which the number of elements grows unpredictably or when flexible insertion and deletion of elements is required. A different approach to organizing data is therefore necessary.

Linked lists provide such a flexible structure. In a linked list, each data element is connected to another through pointers rather than through fixed positions in memory. Each element of the list is combined with a pointer that refers to the next element in the sequence. The combination of data and pointer forms a structure known as a node.

Since nodes are allocated independently in memory, their physical placement is not predetermined. The logical order of the elements is defined exclusively through the pointer references that link one node to the next. This makes it possible to construct lists whose length can change dynamically during program execution.

Depending on the number of pointer references stored in each node, different forms of linked lists can be distinguished. The most common structures are singly linked lists and doubly linked lists.

9.1 Singly Linked Lists

In a singly linked list, each node contains a data element and a single pointer that refers to the next node in the sequence. The first element of the list is accessed through a pointer that marks the beginning of the list. If this pointer contains the value zero, the list is considered empty.

Nodes of a singly linked list are typically created dynamically and connected through their pointer fields. Unlike arrays, the nodes are not required to be stored in contiguous memory locations. Instead, the logical order of elements is defined solely by the pointer connections between nodes.

Accessing elements of the list is performed by following the chain of pointer references starting from the first node. Each node provides the address of the next node until the end of the list is reached. The last node is identified by a pointer whose value is zero.

Several basic operations can be performed on singly linked lists. One common operation is the insertion of a new node at the end of the list. In this case, the list must first be traversed until the last node is found. The pointer of the last node is then updated to reference the new node, while the pointer of the new node is set to zero.

An alternative approach is to insert new nodes at the beginning of the list. This operation is more efficient because the first node can be accessed directly through the list pointer. The new node becomes the new first element, while its pointer refers to the previous first node.

To simplify insertions at the end of the list, an additional pointer may be introduced that always references the last node. With such a pointer, new elements can be appended without traversing the entire list.

Deleting elements from a singly linked list requires updating the pointer of the preceding node so that it references the node following the removed element. This maintains the integrity of the list structure. If the removed node was allocated dynamically, its memory must be released separately.

For greater generality, nodes may contain pointers to arbitrary data rather than embedding the data directly within the node structure. In such cases, the node stores a pointer to the data and a pointer to the next node, allowing the list structure to be used independently of the specific data type.

9.2 Doubly Linked Lists

Singly linked lists allow traversal only in one direction because each node stores a pointer only to its successor. As a result, accessing the predecessor of a node is not possible directly, and operations such as insertion or deletion at arbitrary positions may require additional traversal of the list. These limitations lead to the concept of doubly linked lists.

In a doubly linked list, each node contains two pointer references: one pointing to the next node and another pointing to the previous node. This structure allows the list to be traversed in both directions and simplifies many operations on the list. Because both neighboring nodes of an element are known, insertion and deletion operations can be performed more efficiently than in singly linked lists.

When inserting a node into a doubly linked list, the pointer references of the surrounding nodes must be updated. The new node must reference both its predecessor and its successor, while the predecessor’s next pointer and the successor’s previous pointer are adjusted accordingly. If the insertion occurs at the beginning or end of the list, the pointers representing the first or last node must also be updated.

Deleting a node requires reconnecting its neighboring nodes so that the predecessor of the removed element points to its successor and vice versa. Special attention must be given when the node to be removed is the first or last element of the list. In such cases, the pointers that represent the beginning or end of the list must be modified accordingly. If the node was created using dynamic memory allocation, its memory must be released separately after it has been removed from the list structure.

A common variation of doubly linked lists is the circular list structure. In this configuration, the successor of the last node refers to the first node and the predecessor of the first node refers to the last node. This design removes special boundary cases and allows traversal to continue seamlessly through the list.

To process all elements of the list, traversal typically begins at the first node and continues through the successor pointers until the entire structure has been visited. In circular lists, traversal continues until the starting node is reached again. Such traversal mechanisms allow sequential processing of all nodes, for example when searching, updating, or storing data.

10. Selected String Functions

In the preceding chapters, the fundamental concepts of programming in C were introduced, with particular emphasis on addressing mechanisms, pointer manipulation, and the representation of data in memory. These theoretical foundations provide the necessary background for understanding how character strings are processed in practical applications. Since strings in C are represented as sequences of characters stored in contiguous memory locations, their manipulation is closely related to pointer-based access and memory operations.

The present chapter focuses on several selected string operations that illustrate how common tasks can be implemented using the mechanisms introduced earlier. Instead of presenting a complete string library, the aim is to demonstrate the design principles behind such functions. The functions examined here therefore serve as representative examples that highlight how pointer-based techniques can be used to perform typical text-processing tasks.

Particular attention is given to the copying of character sequences, which is one of the most fundamental operations when working with textual data. Although such functionality is already provided by the standard C library, the detailed examination of its implementation reveals important aspects of function design, memory access, and performance considerations. The examples discussed in this chapter therefore not only illustrate practical programming techniques but also emphasize the relationship between algorithmic formulation and efficient execution.

10.1 Copying Texts

In many programming situations it becomes necessary to duplicate a character string stored in memory. The standard C library provides the function strcpy for this purpose, which copies a sequence of characters from a source location to a destination location. While the use of this function is straightforward, a closer examination of its design reveals several important considerations regarding memory access, parameter passing, and termination conditions.

Before discussing the implementation of the function itself, it is necessary to clarify several conceptual aspects of string copying. A text in C is represented as a sequence of characters that terminates with a null byte. Consequently, the copying process must transfer each character sequentially from the source location to the destination until the terminating null character is encountered. Because the length of the string is not known in advance, the copying operation must be performed iteratively.

A second important consideration concerns the handling of the source and destination memory regions. The data serving as the source must remain unchanged during the copying process, since the function merely reads characters from the source sequence. For this reason, the source argument is commonly declared as constant data, ensuring that the function does not modify the original text. The destination area, on the other hand, must be writable and must provide sufficient storage capacity to accommodate the copied characters, including the terminating null byte.

The design of a general string-copying routine should avoid unnecessary assumptions about the memory locations involved. Instead of specifying a fixed destination area, the function receives both the source address and the destination address as parameters. This approach ensures that the

function can be used in a wide variety of contexts, independent of the particular memory structures used by the calling program.

Another design decision concerns the return value of the function. Although the copying process itself does not require a return value, it is often useful for the function to return the destination address. This allows the result of the copying operation to be used directly in subsequent expressions or function calls, thereby improving the composability of the program.

Taking these considerations into account, the prototype of the string-copying routine can be formulated so that the destination address is provided as the first argument, followed by the source address. The function returns the destination pointer after the copying process has been completed. Internally, the copying operation proceeds character by character, transferring each value from the source to the destination and advancing both addresses until the null terminator is reached.

While this straightforward implementation is functionally correct, further refinements can improve performance. Since the copying operation may involve a large number of characters, the efficiency of the loop structure and the handling of pointer increments can significantly influence the overall runtime. Optimizations therefore aim to reduce the number of instructions executed within the copying loop and to exploit architectural features such as register usage and side effects of increment operations.

Such optimizations demonstrate that the efficiency of a program is not determined solely by the algorithmic structure, but also by the way in which operations are expressed in the source code. By carefully structuring pointer expressions and minimizing unnecessary computations, the runtime behavior of the function can be improved substantially without altering its logical behavior. The example of the strcpy function thus illustrates how even simple operations can benefit from thoughtful design and low-level optimization techniques.

10.2 Comparison of Texts

The comparison of character strings is a fundamental operation in many text processing tasks. One important application is alphabetical ordering, where two strings must be examined character by character until a difference between them is detected. In such comparisons, the characters of both strings are evaluated sequentially, and the ordering is determined by the first position at which the characters differ. If the characters at the current position are identical, the comparison continues with the next pair of characters. The process terminates either when a difference is encountered or when the terminating null character is reached.

In C programming, characters are internally represented by numerical values according to the ASCII character encoding. As a consequence, the comparison of characters can be performed by treating them as integer values. The relative ordering of characters is therefore determined by the numerical values assigned to them. According to the conventions used in ASCII, uppercase letters precede lowercase letters, while digits occupy a range of values distinct from alphabetic characters. The comparison of two characters can therefore be reduced to the evaluation of the difference between their numerical representations.

When two characters are compared numerically, three outcomes are possible. If the first character has a smaller value than the second, the resulting difference is negative. If the first character has a larger value, the difference is positive. If both characters are identical, the difference is zero. For most

comparison routines the absolute magnitude of the difference is irrelevant; only its sign is used to determine the relative ordering of the strings.

The function commonly used for this purpose is strcmp. The routine processes both strings simultaneously by advancing through them character by character. As long as the characters at the current position are identical and not equal to the terminating null character, the comparison continues. When a difference is encountered, the function returns the numerical difference between the characters that caused the termination of the loop. This result indicates whether the first string is lexicographically smaller than, equal to, or greater than the second string.

The implementation of such a routine can be expressed concisely through pointer-based iteration. Two pointers reference the current position within each string and are incremented simultaneously as long as identical characters are encountered. When the loop terminates, the difference between the characters referenced by the two pointers is returned as the function result. This formulation not only reflects the sequential structure of the comparison process but also aligns with the pointer-oriented design principles that characterize efficient C programming.

Care must be taken when formulating loop conditions involving pointer increments. If the increment operation is included directly within the logical expression of the loop condition, unintended side effects may occur because the evaluation order of the logical operators can influence whether the increment is executed. Such constructs may lead to ambiguous behavior and should therefore be avoided in favor of clearer formulations that maintain predictable execution semantics.

Overall, the comparison of character sequences demonstrates how pointer arithmetic and sequential memory access can be combined to implement efficient text-processing operations. By exploiting the numerical representation of characters and the contiguous layout of strings in memory, a compact yet powerful routine can be constructed for lexicographical comparison.

10.3 Character Transformations

Character strings not only represent textual information but can also serve as textual representations of numerical values. In many applications it becomes necessary to convert a sequence of characters that represent digits into a numerical value that can be processed arithmetically. Such transformations form an important bridge between textual input and numerical computation.

A number expressed as a string consists of a sequence of characters that represent digits in a particular number system. For example, the decimal number 123 is represented by the sequence of characters ‘1’, ‘2’, and ‘3’. In order to interpret this sequence numerically, each character must first be converted into its corresponding numeric value. The resulting value is then incorporated into an accumulated sum that reflects the positional weighting of the digits.

The transformation process follows a simple principle. Starting with an initial value of zero, the current numerical value is multiplied by the base of the number system and the numeric value of the next digit is added. This procedure is repeated for each character in the sequence until a character is encountered that does not belong to the permitted range of digits. The validity of characters is therefore determined according to a plausibility principle, which specifies the set of characters that are acceptable for the given number system.

In the case of decimal numbers, only the characters representing digits from ‘0’ to ‘9’ are valid. The conversion routine examines each character sequentially, verifies that it lies within the permitted

range, and then converts it into its numeric equivalent by subtracting the ASCII value of the character ‘0’. The resulting digit value is incorporated into the accumulated sum according to the positional rule described above. The process terminates when a character outside the permitted range is encountered.

A similar approach can be applied to other number systems. For octal numbers, only the digits ‘0’ through ‘7’ are valid, and the accumulated value is multiplied by eight before the next digit is added. Hexadecimal numbers extend the range of valid characters further by including both numeric digits and alphabetic characters representing values from ten to fifteen. In this case, characters in the ranges ‘A’–‘F’ and ‘a’–‘f’ must be interpreted appropriately when computing the numeric value of the digit.

By implementing specialized routines for each number system, character sequences representing decimal, octal, or hexadecimal numbers can be converted into their corresponding numeric values. These routines follow a common structural pattern but differ in the base used for weighting and the validity checks applied to input characters.

In practical applications, it is often useful to provide a general conversion routine that determines automatically which number system is represented by the input string. Such a routine typically examines the first character of the sequence. If the string begins with the character ‘0’, the following character determines whether the value should be interpreted as octal or hexadecimal. A prefix of “0x” or “0X” indicates hexadecimal notation, while a leading ‘0’ without such a prefix indicates octal representation. All other inputs are interpreted as decimal numbers.

Through this mechanism, a single conversion function can interpret numeric strings written in multiple representations. The routine simply delegates the processing to the appropriate conversion function once the number system has been identified. This design demonstrates the flexibility of pointer-based string processing and highlights how simple structural rules can be used to interpret complex textual representations of data.

11. Memory Dump

During the development and testing of programs that manipulate complex data structures, it is often necessary to inspect the contents of memory directly. Such inspection enables programmers to verify whether data structures are stored correctly and to identify errors that may arise during program execution. Instead of embedding numerous diagnostic statements into the program code, a more flexible approach is to use a routine that outputs a selected region of memory to the screen. By examining this output, the programmer can detect inconsistencies in memory layout, incorrect pointer values, or unintended modifications of stored data.

The routine presented in this chapter produces a representation of memory contents in both hexadecimal form and ASCII notation. Each byte of memory is therefore displayed in two different representations: the numerical value in hexadecimal format and the corresponding character representation, provided that the character belongs to the printable range. This dual representation allows both numerical data and textual content to be examined simultaneously, which greatly facilitates debugging tasks.

When programs make extensive use of pointers, structures, or dynamically allocated memory, errors can occur that are difficult to identify by examining program logic alone. For example, incorrect pointer arithmetic may overwrite adjacent memory locations, or an array index may exceed the

allocated bounds of the data structure. In such situations a memory dump provides an immediate view of the affected region and makes it possible to determine where the memory contents begin to deviate from the expected values.

The routine developed in this chapter is designed as a self-contained debugging tool. Although it can be executed as a standalone program, it is structured so that it can easily be integrated into other applications. Instead of placing the entire program directly inside the debugging routine, a small wrapper function containing main is provided that invokes the actual dump function. This approach allows the routine to be reused in different contexts without modification.

11.1 Mode of Operation

The display of memory contents must present a large amount of information in a format that is both compact and readable. In low-level programming environments, hexadecimal notation has proven particularly suitable for representing memory values. Since each hexadecimal digit corresponds to four binary bits, a single byte can be represented using only two characters. This representation allows memory addresses and values to be displayed efficiently without excessive screen space.

For textual interpretation, however, hexadecimal representation alone is not always sufficient. When the memory area contains character strings, it is often more convenient to view the corresponding ASCII representation. In ASCII notation each byte corresponds to a single character, making it easy to recognize words, symbols, or textual patterns stored in memory. For this reason the dump routine displays both hexadecimal values and ASCII characters simultaneously.

A difficulty arises when the memory contents include characters that are not printable. In the ASCII table, values in the range from 0 to 31 and the value 127 correspond to control characters that are used for communication with peripheral devices rather than for textual display. If such characters were printed directly, they could trigger unintended actions such as line feeds or cursor movements. To avoid these effects, the dump routine replaces non-printable characters with a placeholder symbol, typically a dot. As a result, only characters within the printable ASCII range are displayed explicitly, while all other values appear as neutral markers.

The dump output is organized into blocks to improve readability. Each line contains a fixed number of bytes, typically sixteen, displayed in hexadecimal notation. The corresponding ASCII representation is shown alongside the numerical values so that both representations can be inspected simultaneously. After a block of several lines has been displayed, the program waits for user confirmation before continuing with the next block. This mechanism prevents the screen from being flooded with information and allows the user to examine the displayed memory region step by step.

Because memory is accessed byte by byte, the dump routine treats the starting address as a pointer to an unsigned character. This representation ensures that the memory region can be traversed sequentially without interpreting the data according to a specific type. The starting address itself is provided as a parameter, allowing the routine to examine any region of memory specified by the user. In practice, it is advisable to align the starting address to a convenient boundary so that the displayed blocks correspond to natural memory segments.

With these conventions established, the general framework for implementing the dump routine becomes clear. The routine must iterate through a specified range of memory, display each byte in hexadecimal form, translate printable characters into their ASCII representation, and group the

output into readable blocks. Once these design principles have been defined, the implementation of the dump routine can proceed in a systematic manner.

11.2 Solution Approach: Function main

The function main in the memory dump program acts as the central control component that prepares The function main in the memory dump program serves as the control component that prepares the parameters required by the dump routine. Its task is to determine the starting address of the memory region and the format in which the memory contents should be displayed. After these parameters are established, control is transferred to the dump routine.

The program first requests the starting address from the user. Since memory addresses are typically written in hexadecimal form, the address is entered as a hexadecimal character sequence. This sequence is stored in an input buffer and then converted into a numeric value using the routine xtol, which transforms the hexadecimal string into a value of type long. The resulting value represents the starting address of the memory dump and is subsequently converted into a pointer type for memory access.

To ensure a structured output, the starting address is normalized to the nearest lower block boundary. Because the dump routine processes memory in blocks of 256 bytes, the entered address is rounded down to the next lower multiple of 256. This guarantees that the displayed memory blocks begin at a consistent boundary. After the starting address has been determined, the user selects the display format. The dump routine supports several representations corresponding to different data sizes, such as byte, word, or long word. The selected character is translated into an internal numeric code that identifies the format.

Once both parameters have been determined, the function main calls the dump routine and passes the starting address together with the selected display format. At this point the preparation performed by main is complete, and the processing of memory data is handled by the dump function.

11.3 Solution Approach: Function dump

The function dump constitutes the central component of the memory inspection program. While the function main merely determines the required parameters, the dump routine performs the actual formatting and display of the memory contents. Its task is to read the specified memory region sequentially, convert each value into a suitable representation, and output the results in a structured format that facilitates analysis.

The dump routine operates on a block-oriented principle. Instead of displaying the entire memory region in a continuous stream, the output is divided into manageable blocks consisting of several lines. Each line contains a fixed number of bytes, typically sixteen, which are displayed in hexadecimal form. In addition to the numerical values, an ASCII representation of the same bytes is displayed alongside the hexadecimal values. This dual representation enables the programmer to interpret both binary data and textual content directly within the same output.

To produce this structured output, the routine employs two nested loops. The outer loop controls the number of lines within a block, while the inner loop processes the individual bytes of a single line. For each byte, the routine reads the corresponding memory value and converts it into its hexadecimal representation using formatted output functions. At the same time, the byte is evaluated to

determine whether it corresponds to a printable ASCII character. If the value lies within the printable range of the ASCII table, the character itself is displayed; otherwise a placeholder symbol is used.

For reasons of efficiency and clarity, the formatted output is first assembled within a line buffer before it is printed to the screen. Instead of calling the output routine separately for each individual byte, the function constructs an entire line of output using the function sprintf. This function behaves similarly to printf, but writes the formatted text into a character array rather than directly to the display. Once the entire line has been prepared in the buffer, it can be printed with a single output operation. This approach reduces screen flicker and improves the readability of the generated output.

At the beginning of each output line, the current memory address is displayed so that the programmer can easily identify the location of the data being examined. After a line has been printed, the address pointer is advanced by the number of bytes processed in that line. In this way the dump routine progresses sequentially through the memory region, presenting the contents in clearly organized segments.

After the completion of each block of lines, the routine pauses and waits for user input before continuing with the next block. This interactive mechanism allows the user to inspect the displayed data carefully without the output scrolling too quickly across the screen. If the user enters a specific termination character, the routine stops and returns control to the calling program.

Through this structured design, the dump routine provides a flexible and effective tool for examining memory contents during program development. By combining hexadecimal representation, ASCII interpretation, and block-based output formatting, it enables programmers to identify irregularities in memory structures and to diagnose errors that might otherwise remain hidden within complex programs.

12. Address Management

Computers are used not only for performing calculations but also for organizing and managing information. In many applications, large collections of structured data must be stored, retrieved, and modified efficiently. The program presented in this chapter demonstrates how such data management tasks can be implemented in C using dynamic data structures and modular program design.

The example application manages personal records containing information such as name, address, telephone number, and date of birth. Each record is stored within a dynamic data structure so that the number of entries is not fixed in advance. As long as memory is available, new records can be inserted into the data set. This flexibility makes the structure suitable for practical data management tasks.

Interaction with the program is performed through a menu-based interface. The user can perform several operations on the stored data, including reading records from a file, storing records in a file, adding new records, deleting existing records, sorting the list of records, displaying stored data, and modifying previously entered information. These operations represent the typical functionality required in a simple record management system.

To organize the stored information efficiently, the program uses a doubly linked list structure. Each node of the list contains a pointer to the actual data record as well as references to the previous and the next node in the list. This structure allows records to be inserted or removed without shifting

large blocks of memory, which would be necessary if the records were stored in a contiguous array. As a result, list operations remain efficient even when the data set grows dynamically.

The data records themselves are defined using structures. Several fields are included in each record in order to represent the personal information of an individual. For example, postal codes are stored independently from the place of residence and may include an optional country identifier. Telephone numbers are represented as character strings so that different formatting styles can be supported. The date of birth is stored in a structured form containing separate components for day, month, and year.

In addition to the data structures, a number of symbolic constants are introduced using preprocessor directives. These constants represent menu selections, error states, and other control values used throughout the program. By assigning meaningful names to these values, the readability of the program is improved and modifications can be performed more easily.

The main program controls the overall operation of the application. It repeatedly displays a menu and executes the corresponding function selected by the user. The menu system returns a value indicating the selected operation, and the main program processes this value using a switch statement. Each case in this statement invokes the appropriate function responsible for performing the requested operation.

The output of informational messages to the user is handled by a dedicated function. This function displays short texts describing program states or error conditions and waits for a user response before execution continues. In order to keep the program portable, terminal-specific control sequences are not embedded directly into the output statements. Instead, all terminal-dependent functionality is isolated in separate modules so that the program can be adapted easily to different systems.

Displaying the menu requires clearing the screen and presenting the available operations in a structured format. At the top of the screen, the program also displays the number of records currently stored in the list. If the list is empty, an appropriate message is shown instead. After displaying the menu, the program reads a character from the keyboard and interprets it as the selected menu option.

Individual data records can be displayed using a dedicated output routine. This function formats the contents of a record and prints the values in a readable layout. Since the node of the linked list already contains a pointer to the stored data record, no additional address calculations are required. The function simply accesses the individual fields of the structure and outputs them to the screen.

When a new record is inserted, a new node must first be created in the linked list. Memory for this node is allocated dynamically using the function malloc. If sufficient memory is available, the new node is inserted into the list and connected to the neighboring nodes. This process ensures that the structural integrity of the list is maintained.

After a new node has been created, the fields of the associated data record must be filled with values provided by the user. The program prompts the user for each field individually. Existing values may be displayed as defaults so that the user can accept them or replace them with new input. Character strings are copied using safe string handling functions to avoid buffer overflow, and numerical values entered by the user are converted into integer representations using standard conversion routines.

In order to prevent accidental loss of information, the program provides functionality for storing the entire list of records in an external file. During this process, all records are written sequentially using

standard file input and output operations. The program checks whether the list contains data and whether changes have been made since the last save operation before initiating the file writing procedure.

Saving records to a file is performed using binary output functions. Each data record is written exactly as it is stored in memory, which simplifies both the writing and the later reading of the data. If an error occurs during the file operation, the program returns an appropriate status value so that the calling function can react accordingly.

In addition to storing records, the program also supports deletion of existing entries. Records may either be removed individually or the entire list may be cleared at once. When deleting nodes from the list, it is essential to release the dynamically allocated memory using the function free. This prevents memory leaks and ensures that the program does not gradually consume available memory during long execution periods.

Selective deletion requires traversing the list and asking the user whether a particular record should be removed. If the user confirms the deletion, the corresponding node is removed from the list and its associated memory is released. When the entire list is deleted, the program iterates through all nodes and frees each one in sequence until the list becomes empty.

Records can also be loaded from a previously saved file. When reading records from a file, the program reconstructs the list by creating a new node for each record read from the file. If records already exist in memory, the user is asked whether the existing list should be deleted before loading the new data. This precaution prevents unintentional mixing of old and newly loaded data.

When a larger number of records is stored in the system, it becomes useful to sort the list according to specific criteria. The program therefore includes a sorting routine that orders the records based on the contents of selected fields such as name or place of residence.

The sorting operation is implemented using a simple bubble sort algorithm. Although more efficient algorithms exist, bubble sort is easy to implement and sufficiently efficient for relatively small data sets. Instead of exchanging the entire data structures, the algorithm swaps pointers to the records stored within the nodes. This approach significantly reduces the amount of memory copying required during sorting.

During the sorting process, the comparison of two records is performed by a dedicated comparison function. For textual fields, standard string comparison routines are used to determine the alphabetical order of the records. Numerical fields are compared using simple arithmetic comparisons.

Finally, the program allows existing records to be modified. The user selects a record from the list and the program displays its current contents. Individual fields can then be updated using the same input routines that were used during the initial data entry process. This ensures that data formatting and validation remain consistent throughout the application.

Through this example, the chapter demonstrates how dynamic memory management, structured data types, linked lists, and modular program organization can be combined to implement a complete record management system in the C programming language. The program illustrates how larger applications can be developed incrementally from smaller reusable components while maintaining portability and clarity of structure.